CASE STUDY 2

Unit II: Searching and Sorting / Arrays

Social Network Friendship Connections Using an Adjacency Matrix

Aim

To represent users and their friendship connections using an adjacency matrix.

Objectives

- Represent users as vertices.
- Store whether two users are connected.
- Check friendship connections using a matrix.

Theory

A social network can be represented as a graph. Each user is a vertex and each friendship is an edge. An adjacency matrix is a two-dimensional list. A value of 1 means that two users are connected, while 0 means that they are not. For an undirected friendship graph, the matrix is symmetric.

Algorithm

1. Create a list of users.
2. Create a square matrix filled with zeroes.
3. For every friendship, place 1 in both corresponding matrix positions.
4. Use the matrix to check whether two users are friends.

Python Program

```
# Social network using an adjacency matrix
users = ["A", "B", "C", "D"]
matrix = [[0 for j in users] for i in users]

friendships = [("A", "B"), ("B", "C"), ("A", "D")]

for person1, person2 in friendships:
    i = users.index(person1)
    j = users.index(person2)
    matrix[i][j] = 1
    matrix[j][i] = 1

print("  ", *users)
for i in range(len(users)):
    print(users[i], matrix[i])

first = "A"
second = "B"
i = users.index(first)
j = users.index(second)

if matrix[i][j] == 1:
    print(first, "and", second, "are friends")
else:
    print(first, "and", second, "are not friends")
```

Sample Output

```
    A B C D\nA [0, 1, 0, 1]\nB [1, 0, 1, 0]\nC [0, 1, 0, 0]\nD [1, 0, 0, 0]\nA and B are friends
```

Conclusion

An adjacency matrix is a straightforward way to store and check friendship connections in a graph.